

Kit 3 - Sprint 3



Projet smartEngine

Prédiction de churn pour un SaaS B2B

Modules

Scoring prédictif et Marketing

Ciblage et Data Marketing

Préparé par Nejma HAMMOUTOU

1. Rappel et bilan du Sprint 2

1.1 Où en sommes-nous

Au Sprint 2, vous avez nettoyé les données brutes de RavenStack et construit une table analytique unique avec une ligne par compte client.

Cette table (`data/processed/analytics.csv`) contient des features et la variable cible churn (0 ou 1).

Le Sprint 3 est le cœur du projet : c'est ici que vous construisez le modèle de scoring prédictif qui va produire les scores de churn pour chaque compte. Ce modèle sera ensuite déployé dans un dashboard au Sprint 4.

1.2 Rotation des rôles

Les rôles Product Owner et Scrum Master tournent de nouveau. Les deux membres qui n'ont pas encore occupé ces postes prennent le relais. Mettez à jour votre backlog et votre GEMINI.md avec les nouveaux rôles.

1.3 Ce qui doit être fait avant de commencer

Avant de démarrer les travaux du Sprint 3, vérifiez que :

- Tous les livrables du Sprint 2 sont dans la branche main (table analytique, rapport de nettoyage, scripts, section 2 du dossier de conception)
- Le fichier `data/processed/analytics.csv` est accessible sur toutes les machines
- Le GEMINI.md est à jour avec le sprint en cours
- Le compte-rendu de la sprint review du Sprint 2 est déposé dans `docs/standups/`

2. Objectifs du Sprint 3

L'objectif de ce sprint est de transformer votre table analytique en un modèle de scoring prédictif opérationnel. À la fin du sprint, vous disposez :

- D'un modèle entraîné et sauvegardé (`outputs/models/churn_model.joblib`)
- D'un fichier de scores pour tous les comptes (`outputs/scores.csv`) avec une probabilité de churn et un niveau de risque par compte
- D'un rapport de performance documenté qui justifie vos choix et analyse les limites du modèle

Le sprint se décompose en 5 étapes :

- Préparation des données pour la modélisation (split train/test, gestion du déséquilibre)
- Choix et entraînement de plusieurs algorithmes
- Évaluation et comparaison des modèles
- Interprétation et analyse des biais
- Génération des scores et sauvegarde du modèle

C'est aussi le sprint où vous créez un nouvel agent dédié à la modélisation, où vous enrichissez votre agent data-engineer existant.

2.1 Evolution de vos agents

Au Sprint 2, vous avez un agent data-engineer qui nettoie et transforme les données. Pour le Sprint 3, les tâches sont différentes : entraînement d'un modèle, évaluation statistique, interprétation.

Deux approches :

- Créer un nouvel agent dédié : model-trainer.md spécialisé dans la **modélisation supervisée**.
Approche modulaire, plus lisible
- Enrichir votre agent existant : ajouter les instructions de modélisation à data-engineer.md.

Justifiez votre choix dans le dossier de conception.

Livrable : Agent de modélisation (nouveau ou enrichi) .gemini/agents/[nom].md avec frontmatter (name, description) et corps structure (rôle, étapes, règles). Le choix est justifié dans le dossier de conception.

Contrainte importante : à la livraison finale, le projet ne contiendra pas Gemini CLI. Tous les scripts produits dans ce sprint (train_model.py, evaluate_model.py, generate_scores.py) doivent pouvoir s'exécuter de façon autonome, en ligne de commande, sans dépendre de l'outil d'orchestration. Structurez-les dès maintenant pour qu'ils s'enchaînent.

3. Mise à jour de l'infrastructure

3.1 Mise à jour du GEMINI.md

Votre GEMINI.md doit refléter l'avancement du projet. Mettez-le à jour avec :

- Le sprint en cours (Sprint 3)
- Les nouveaux rôles (nouveau SM, nouveau PO)
- Un résumé de ce qui a été fait au Sprint 2 (table analytique, rapport de nettoyage)
- Le chemin vers les livrables à produire : outputs/models/churn_model.joblib et outputs/scores.csv

3.2 Structure du dépôt à la fin du Sprint 3

```
smartengine-groupe-X/
|-- .gitignore
|-- .gemini/
|   |-- agents/
|       |-- data-explorer.md    # Sprint 1
|       |-- data-engineer.md    # Sprint 2
|       |-- model-trainer.md    # NOUVEAU Sprint 3
|-- data/
|   |-- raw/
|   |-- processed/
|   |-- analytics.csv
|-- outputs/
|   |-- rapport-nettoyage.md    # Sprint 2
|   |-- rapport-modele.md      # NOUVEAU Sprint 3
```

```
| |-- scores.csv          # NOUVEAU Sprint 3
| |-- models/            # NOUVEAU Sprint 3
| |-- churn_model.joblib
|-- src/
| |-- clean_data.py      # Sprint 2
| |-- build_features.py  # Sprint 2
| |-- build_analytics.py # Sprint 2
| |-- train_model.py     # NOUVEAU Sprint 3
| |-- evaluate_model.py  # NOUVEAU Sprint 3
| |-- generate_scores.py  # NOUVEAU Sprint 3
|-- docs/
| |-- standups/
| |-- dossier-conception.docx # section 3 ajoutée
| |-- ...
|-- GEMINI.md
`-- README.md
```

4. Travaux du Sprint 3

4.1 Préparation des données pour la modélisation

Avant d'entraîner un modèle, il faut préparer les données. Cette étape conditionne la qualité de tout ce qui suit.

Split train / test

Vous ne pouvez pas entraîner et évaluer votre modèle sur les mêmes données. Si vous le faites, le modèle peut apprendre par cœur les exemples et avoir une performance artificiellement élevée. On sépare donc le dataset en deux :

- **80%** pour entraîner le modèle -> **train**
- **20%** pour l'évaluer sur des cas qu'il n'a jamais vus -> **test**

Utilisez `train_test_split` de `scikit-learn` avec `stratify=y` pour conserver les mêmes proportions de churn dans le train et le test. Fixez `random_state` pour la reproductibilité.

Gestion du déséquilibre des classes

Dans une base SaaS B2B, la grande majorité des clients restent actifs sur la fenêtre observée. Vos churners sont donc minoritaires, parfois très minoritaires. Avant de lancer le moindre entraînement, calculez le ratio churn / non-churn sur votre table d'entraînement et notez-le dans votre rapport. C'est ce ratio qui détermine la suite des choix.

Quand une classe écrase l'autre, un modèle peut atteindre un score d'accuracy* très élevé en prédisant simplement « non-churn » pour tout le monde. Sur un jeu où peu de clients churnent, prédire toujours la classe majoritaire donne une accuracy en apparence excellente sans détecter de cas à risque, donc sans aucune valeur métier. C'est exactement l'inverse de ce qu'on attend, l'enjeu, c'est d'identifier les churners.

Il faut donc forcer le modèle à prêter attention à la classe minoritaire. Trois stratégies sont à comparer :

- **class_weight='balanced'** dans le modèle
- **SMOTE** (via imbalanced-learn)
- **Undersampling** : réduire la classe majoritaire. Perte d'information

Le choix est libre mais doit être justifié dans le dossier de conception.

Tache de recherche

Avant de préparer vos données, documentez dans votre base de connaissances personnelle :

- Qu'est-ce qu'un modèle prédictif ? Comment apprend-il à partir de données ?
- Quelle est la différence entre apprentissage supervisé et non supervisé ? Dans quelle catégorie se situe smartEngine et pourquoi ?
- Qu'est-ce qu'un problème de classification ? En quoi diffère-t-il d'une régression ?
- Pourquoi sépare-t-on les données en *train* et *test* ? Quelle proportion utiliser ?
- Qu'est-ce que l'**overfitting** ? Comment le détecter ?
- Qu'est-ce que la validation croisée (k-fold) ? Quand est-elle utile ?
- Qu'est-ce qu'un dataset déséquilibré ? Pourquoi est-ce un problème ?

4.2 Choix et entraînement du modèle

Vous allez entraîner plusieurs algorithmes et comparer leurs performances. L'objectif n'est pas d'en choisir un au hasard mais de comprendre pourquoi un algorithme convient mieux à votre problème.

Les algorithmes à tester

Entraînez au moins trois algorithmes différents pour pouvoir les comparer :

- **Logistic Regression** : le plus simple, rapide, très interprétable, bonne baseline
- **Random Forest** : combinaison d'arbres de décision, robuste, gère bien les données hétérogènes
- **Gradient Boosting (XGBoost ou LightGBM)** : souvent le plus performant mais plus complexe

Utilisez [scikit-learn](#) pour les deux premiers. XGBoost et LightGBM sont des librairies séparées à installer.

Les hyperparamètres

Chaque algorithme a des paramètres qui influencent sa performance. Au Sprint 3, vous pouvez commencer avec les valeurs par défaut. Si le temps le permet, utilisez GridSearchCV ou RandomizedSearchCV pour trouver les meilleures valeurs.

RandomizedSearchCV et GridSearchCV cherchent automatiquement les meilleurs hyperparamètres d'un modèle en testant plusieurs combinaisons et en gardant celle qui obtient le meilleur score en validation croisée.

Tache de recherche

Avant de choisir votre algorithme, documentez :

- Comment fonctionne chacun des 3 algorithmes (Logistic Regression, Random Forest, Gradient Boosting) en une phrase
- Quels sont les avantages et les limites de chacun ?
- Quel critère prime pour smartEngine : performance brute ou interprétabilité ? Pourquoi ?

4.3 Evaluation du modèle

Une fois les modèles entraînés, il faut les évaluer. Un modèle qui a l'air performant peut être inutilisable en production si on choisit les mauvaises métriques.

La matrice de confusion

La matrice de confusion est le point de départ de toute évaluation en classification. Elle montre pour chaque combinaison prédiction/réalité le nombre de cas :

- Vrais positifs (**VP**) : le modèle a prédit churn, c'était un churn. **OK**
- Faux positifs (**FP**) : le modèle a prédit churn, ce n'était pas un churn. **On dérange un client fidèle**
- Vrais négatifs (**VN**) : le modèle a prédit non-churn, ce n'était pas un churn. **OK**
- Faux négatifs (**FN**) : le modèle a prédit non-churn, c'était un churn. **On a raté un départ**

Les métriques

Quatre métriques à calculer systématiquement :

- **Accuracy** : % de prédictions correctes. Attention, trompeuse avec des classes déséquilibrées
- **Precision** : parmi les comptes prédits comme churn, combien l'étaient vraiment
- **Recall** : parmi les vrais churns, combien on a détecté
- **F1-score** : moyenne harmonique de precision et recall, bon compromis
- **AUC-ROC** : capacité du modèle à distinguer les classes sur différents seuils

L'accuracy figure dans la liste pour une raison précise : vous devez la calculer et la comparer à votre F1-score sur la classe churn. C'est l'écart entre les deux qui rend visible l'effet du déséquilibre. Une accuracy très haute couplée à un F1 très bas est le signal qu'un modèle prédit majoritairement « non-churn » sans détecter les cas qui nous intéressent.

Pour RavenStack, qu'est-ce qui coûte le plus cher, rater un churn ou alerter à tort ? Déduisez-en la métrique que vous devez privilégier, et justifiez votre choix dans le rapport de modélisation.

Tache de recherche

Avant d'évaluer vos modèles, documentez :

- Qu'est-ce qu'une matrice de confusion ? Comment la lire ?
- Définissez *precision*, *recall* et *F1-score*. Dans quel cas privilégier l'un plutôt que l'autre ?
- Pourquoi l'*accuracy* est-elle trompeuse quand les classes sont déséquilibrées ? Illustrez avec les chiffres de RavenStack
- Qu'est-ce que l'*AUC-ROC* ? Comment interpréter cette métrique ?

4.4 Interprétation et analyse des biais

Un modèle qui prédit sans expliquer est inutilisable en entreprise. L'équipe doit comprendre POURQUOI un compte est flaggé pour adapter son action.

l'article 22 du RGPD impose un droit à l'explication pour toute décision automatisée.

Feature importances

Pour un Random Forest ou un Gradient Boosting, scikit-learn fournit directement l'importance de chaque feature : quelles variables ont le plus compté dans les décisions du modèle. Permet de vérifier que le modèle s'appuie sur des signaux cohérents avec l'intuition métier.

Exemple : si `usage_trend` arrive en tête, c'est cohérent avec l'hypothèse qu'un désengagement progressif précède le churn. Si c'est `account_name` qui sort (improbable ici), il y a un problème.

SHAP pour expliquer une prédiction individuelle

SHAP (SHapley Additive exPlanations) va plus loin : pour chaque compte, il donne la contribution de chaque feature à la prédiction. Permet au CSM de savoir précisément pourquoi un compte spécifique est à risque.

Analyse des biais

Un modèle peut être techniquement bon et métier biaisé. Exemples de biais possibles dans smartEngine :

- **Biais par industrie** : si DevTools est surreprésenté dans les churns d'entraînement, le modèle peut sur-prédire le churn pour cette industrie
- **Biais par pays** : calculez la répartition par pays dans votre table avant de chercher des biais. Le modèle peut moins bien fonctionner pour les autres pays
- **Biais par plan** : les *Enterprise* sont-ils aussi bien prédits que les *Basic* ?

Comment les détecter : évaluez les métriques (recall, precision) par sous-groupe. Si le recall est à 80% en moyenne mais 40% sur les comptes français, il y a un problème.

Tache de recherche

Avant d'interpréter vos modèles, documentez :

- Qu'est-ce que l'interprétabilité d'un modèle ? Pourquoi est-elle cruciale en B2B et vis-à-vis du RGPD ?
- Que sont les *feature importances* ? A quoi servent-elles pour les équipes ?
- Qu'est-ce que **SHAP** ? Quelle différence avec les *feature importances* classiques ?
- Qu'est-ce qu'un biais algorithmique ? Donnez un exemple concret qui pourrait toucher smartEngine
- Comment détecter et atténuer un biais ?
- En quoi l'article 22 du RGPD fait de l'interprétabilité une exigence réglementaire et pas seulement un confort technique ?

4.5 Sauvegarde et génération des scores

Sauvegarde du modèle

Une fois le meilleur modèle identifié, sauvegardez-le avec joblib, qui est le format standard recommande par scikit-learn :

```
import joblib
joblib.dump(model, 'outputs/models/churn_model.joblib')
```

Vérifiez la taille du fichier avant de committer. Au-delà de 100 Mo, ajoutez le dossier au `.gitignore`.

Génération des scores pour tous les comptes

Le modèle entraîné sert à produire un score de churn pour chaque compte. C'est ce fichier qui alimentera le dashboard au Sprint 4.

Contenu attendu de `outputs/scores.csv` :

- `account_id` : identifiant du compte
- `churn_score` : probabilité de churn (entre 0 et 1)

- `risk_level` : niveau de risque (high / medium / low) selon les seuils que vous définissez

Le choix des seuils (par exemple 0.7 et 0.4) est un choix métier. Justifiez-le dans le dossier de conception.

Tache de recherche

Documentez :

- Qu'est-ce qu'un seuil de décision ? Comment le choisit-on ?
- Pour RavenStack, comment définir les niveaux high / medium / low ? Quel intérêt pour l'équipe Customer Success ?

5. Dossier de conception : section 3

Ajoutez une troisième section à votre dossier de conception (docs/dossier-conception.docx). Cette section couvre la modélisation prédictive.

5.1 Contenu attendu

La section 3 doit répondre aux questions suivantes :

- Choix d'algorithme : quels algorithmes avez-vous testé ? Lequel avez-vous retenu et pourquoi ? Quelle alternative avez-vous écarté et pourquoi ?
- Préparation des données : comment avez-vous géré le déséquilibre des classes ? Quelle stratégie de split ?
- Evaluation : quelles métriques avez-vous suivies et pourquoi ? Quels sont les résultats sur le set de test ?
- Interprétation : quelles sont les features les plus importantes ? Est-ce cohérent avec l'intuition métier ?
- Biais : avez-vous identifié des biais ? Comment les avez-vous mesurés ? Comment comptez-vous les atténuer ?
- Limites : quelles sont les limites connues de votre modèle ? Dans quels cas fonctionne-t-il moins bien ?
- Choix des seuils : pourquoi ces seuils pour high / medium / low ?

Livrable : Section 3 du dossier de conception, docs/dossier-conception.docx. Chaque choix technique est justifié en termes métier, pas seulement technique.

6. Backlog Sprint 3

Mettez à jour votre backlog avec les tâches du Sprint 3. Les user stories du Sprint 2 doivent passer au statut 'Done'.

Exemples de user stories Sprint 3 :

- En tant que développeur IA, je veux entraîner au moins 3 algorithmes différents afin de pouvoir comparer leurs performances.
- En tant que Product Owner, je veux un modèle avec un recall > 75% sur la classe churn afin de détecter un maximum de comptes à risque.
- En tant que Scrum Master, je veux un rapport de performance documenté afin de garantir la traçabilité et la conformité RGPD.

Livrable : Backlog Sprint 3, même outil qu'aux sprints précédents. Stories Sprint 2 passées en Done, nouvelles stories Sprint 3 ajoutées et assignées.

7. Rituels Scrum

7.1 Daily Standup

10 minutes maximum en début de séance. Trois questions : qu'ai-je fait, que vais-je faire, qu'est-ce qui me bloque. Compte-rendu dans docs/standups/AAAA-MM-JJ.md.

7.2 Sprint Review

A la fin du sprint, vous présentez :

- Le modèle retenu et ses performances (matrice de confusion, métriques)
- Les features les plus importantes et leur interprétation métier
- Les biais identifiés et les limites du modèle
- Le fichier de scores et les seuils retenus
- La section 3 du dossier de conception

8. La suite

Au Sprint 4, vous combinerez le score prédictif produit ici avec une segmentation pour construire un dashboard Streamlit actionnable pour les équipes. Le fichier outputs/scores.csv sera la source principale du dashboard.

9. Ressources

Ressource	Lien
Documentation scikit-learn	scikit-learn.org/stable/
Guide classification scikit-learn	scikit-learn.org/stable/supervised_learning.html
Gestion du déséquilibre	imbalanced-learn.org
Documentation SHAP	shap.readthedocs.io
Sauvegarde de modèles	scikit-learn.org/stable/model_persistence.html
XGBoost	xgboost.readthedocs.io
Métriques de classification	scikit-learn.org/stable/modules/model_evaluation.html
Dataset RavenStack	Discord #datasets

10. Livrables

Livrable	Fichier attendu	Responsable
Agent de modélisation (nouveau ou enrichi)	.gemini/agents/[nom].md	Developpeurs IA
Script d'entrainement	src/train_model.py	Généré par l'agent
Script d'évaluation	src/evaluate_model.py	Généré par l'agent
Script de génération des scores	src/generate_scores.py	Généré par l'agent
Modèle entraîné sauvegardé	outputs/models/churn_model.joblib	Toute l'équipe
Fichier des scores par compte	outputs/scores.csv	Toute l'équipe
Rapport de performance	outputs/rapport-modele.md	Toute l'équipe
Section 3 dossier de conception	docs/dossier-conception.docx	Toute l'équipe
GEMINI.md mis à jour	GEMINI.md (racine)	Scrum Master
Backlog Sprint 3	Lien outil de gestion	Scrum Master